

M Ű E G Y E T E M 1 7 8 2

BUDAPESTI MŰSZAKI ÉS GAZDASÁGTUDOMÁNYI EGYETEM
VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR
MESTERSÉGES INTELLIGENCIA ÉS RENDSZERTERVEZÉS TANSZÉK

Jelgenerátor megvalósítása

Intelligens beágyazott rendszerek laboratórium

(BMEVIMIMA21)

Készítette:

Pál Patrik, Gavallér Dávid

AIQTQW, ES02RO

BUDAPEST, 2026. november 12.

Tartalomjegyzék

1. Bevezetés **2**

2. Függvénygenerátor megvalósítása **3**

1. Bevezetés

A mérés célja

A feladat során egy olyan jelgenerátort valósítottunk meg, ami képes szinusz-, fűrész- és négyszögjel generálására. A generált jelek frekvenciája állítható, így a mérés során különböző jelalakok és paraméterek vizsgálatára nyílt lehetőség.
A mérés során lehetőségünk volt megismerkedni a jelgenerátorok működésével, különböző jelalakok előállításának gyakorlati módjaival és a UART kommunikáció felhasználásával beágyazott környezetben.

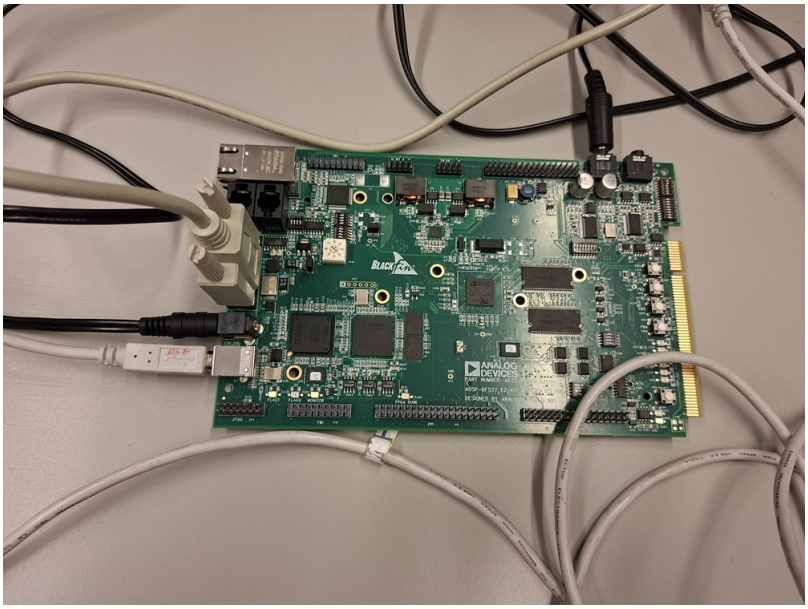
A méréshez használt eszközök

- Oszcilloszkóp
- Számítógép
- CrossCore Embedded Studio
- Blackfin panel

2. Függvénygenerátor megvalósítása

Mérési elrendezés

Első lépésben a Blackfin panelt és az oszcilloszkópot kötöttük össze, majd a Blackfin panelt kapcsoltuk a számítógéphez a JTAG és az RS-232 kábellel. (1. ábra)



1. ábra. A mérési elrendezés.

Feladat megoldása

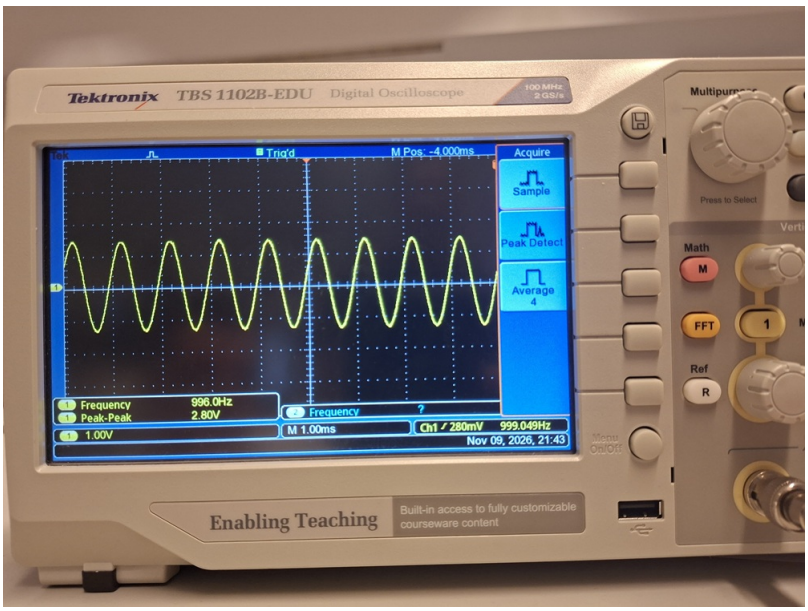
Első lépésben egy előre meghatározott frekvenciájú szinussz jel legenerálása volt a cél. A kezdeti fázist 0-nak választottuk, és minden időlépésben növeltük egy deltaPhase értékkel, amit az alábbi összefüggéssel határoztunk meg:

$$deltaPhase = rbits\left(\frac{(outFreq << 15)}{sampleFreq}\,\&\,mask\right) \tag{1}$$

A kártya 48 kHz mintavételi frekvenciával rendelkezik, ezért a sampleFreq értékét 48000-ra állítottuk. A mask értéke pedig 32767 volt, hogy a 15 bites tartományon belül maradjunk.

Amikor a fázis értékének növelése után telítettük a fract értéktartományát, akkor kiszámítottuk mennyit kellene még növelnünk az aktuális fázis értéken, ahhoz hogy elérjük a `FRACT_MAX`-ot, majd a 0-ról indulva a deltaPhase és ennek a phaseRemaining értékek a különbségére léptettük a fázist, ezzel megoldva azt, hogy ne legyen töréspontunk az 1-ről 0-ra wrappelés során.

A szinusz jelet a `sin2pi_01_fx16` függvény segítségével generáltuk, ami a fázis értékét használva adta vissza a megfelelő szinusz értéket. Az így kapott értéket a `process_data` függvényben a iChannel0RightOut változóba mentettük, ami így már oszcilloszkópon is láthatóvá vált, ahogy a 2. ábra mutatja.



2. ábra. A szinuszjel.

Ezt követően az egyéb jelalakokat valósítottuk meg a `sin2pi_fx.c` fájlban. A négyszög jel kirajzolása esetén, ha az aktuális fázis érték kisebb, mint a `FRACT_MAX/2`, akkor a kimeneti érték `FRACT_MAX`, egyébként pedig `FRACT_MIN`. A fűrészjel esetén pedig a kimeneti érték a fázis értékével egyezett meg.

Az egyes jelalakok közötti váltást UART kommunikáció segítségével valósítottuk meg. A soros porton csatlakoztattuk a számítógépet a Blackfin panelhez, és különböző billentyűket használtunk a jelalak kiválasztásához, egy switch struktúra segítségével.

A jelalak kiválasztását követően a kimeneti jel alapértelmezetten 1000 Hz frekvenciával generálódik, de a felhasználó a soros porton keresztül ezt tudja módosítani. A frekvencia léptethető 1 Hz, 10 Hz, 100 Hz és 1000 Hz értékekkel, a felhasználó által megadott karakterek alapján. Az 'e' billentyű lenyomásával pedig a program kilép, és kiválasztható a jelalak újra a fentebb leírt módon.

Függelék

Felhasznált C kódok

process_data.c

```
#include "frame.h"
#include "sin2pi_fx.h"
#include "stdio.h"
#include <stdint.h>
#include <inttypes.h>

#define N 401                   // number of filter coefficients
```

```
section("L1_data_b") fract coefs[N] = {
#include "bp_fract.dat"
};

fract section("L1_data_a") input[N];
fract output[N];
accum out = 0;
volatile fract outVal;
int i = 0;

fract konv;
uint16_t *konvi;
uint32_t konvi32;

uint32_t outFreq = 1000;
uint32_t sampleFreq = 48000;

fract phase = 0;
fract phaseOld = 0;
fract deltaPhase = 0;
uint32_t mask = 32767;
fract phaseRemaining = 0;
char* initMessage = "The system is in initialization mode, choose a
   signal type\n\r";
char* sinMode = "Sine mode chosen, pick a frequency, current frequency
   is: ";
char* squareMode = "Square mode chosen, pick a frequency, current
   frequency is: ";
char* sawMode = "Sawtooth mode chosen, pick a frequency, current
   frequency is: ";
char* actualFreqMsg = "The actual frequency is: ";
uint8_t state = 0; //0 - init, 1 - sine, 2 - square, 3 - sawtooth
//-----------------------------------------------------------------------//

// Function:    Process_Data()
                                                                      //
//

                            //
// Description: This function is called from inside the SPORT0 ISR
   every        //
//                                time a complete audio frame has been
   received. The new          //
```

```
41 //                              input samples can be found in the
   variables iChannel0LeftIn,//
42 //                              iChannel0RightIn, iChannel1LeftIn and
   iChannel1RightIn          //
43 //                              respectively. The processed     data
   should be stored in               //
44 //                              iChannel0LeftOut, iChannel0RightOut,
   iChannel1LeftOut,          //
45 //                              iChannel1RightOut, iChannel2LeftOut and
    iChannel2RightOut       //
46 //                              respectively.

           //
47 //-------------------------------------------------------------------------//

48 void Process_Data(void)
49 {
50         //115200 baud
51         deltaPhase = rbits(((outFreq<<15) / sampleFreq) & mask);
52         if (uart_rx_flag)
53         {
54                 uart_rx_flag = false;
55                 //uart_putch(uart_getch());
56                 if (state == 0){
57                         switch (uart_getch()){
58                                 case 's':
59                                         state = 1;
60                                         char buffer[128];
61                                         snprintf(buffer, sizeof(buffer)
                                            , "%s%u%s" , sinMode,
                                            outFreq, "\r\n");
62                                         char* szoveg = buffer;
63                                         uart_print(szoveg);
64                                         break;
65                                 case 'q':
66                                         state = 2;
67                                         char buffer2[128];
68                                         snprintf(buffer2, sizeof(
                                            buffer2), "%s%u%s" ,
                                            squareMode, outFreq, "\r\n")
                                            ;
69                                         char* szoveg2 = buffer2;
70                                         uart_print(szoveg2);
```

```
                                        break;
                                case 't':
                                        state = 3;
                                        char buffer3[128];
                                        snprintf(buffer3, sizeof(
                                           buffer3), "%s%u%s" , sawMode
                                           , outFreq, "\r\n");
                                        char* szoveg3 = buffer3;
                                        uart_print(szoveg3);
                                        break;
                                default:
                                        uart_print(initMessage);
                                        break;
                        }
                        }
                        if (state != 0){
                                switch (uart_getch()){
                                case 'h':
                                        outFreq -= 1;
                                        break;
                                case 'z':
                                        outFreq += 1;
                                        break;
                                case 'j':
                                        outFreq -= 10;
                                        break;
                                case 'u':
                                        outFreq += 10;
                                        break;
                                case 'k':
                                        outFreq -= 100;
                                        break;
                                case 'i':
                                        outFreq += 100;
                                        break;
                                case 'l':
                                        outFreq -= 1000;
                                        break;
                                case 'o':
                                        outFreq += 1000;
                                        break;
                                case 'e':
                                        state = 0;
```

```
112                                 uart_print(initMessage);
113                         }
114                         if (outFreq < 0)
115                                 outFreq = 0;
116                         if (state != 0){
117                                 char buffer4[128];
118                                 snprintf(buffer4, sizeof(buffer4), "%s%
                                       u%s" , actualFreqMsg, outFreq, "\r\n
                                       ");
119                                 char* szoveg4 = buffer4;
120                                 uart_print(szoveg4);
121                         }
122                 }
123         }
124         iChannel0LeftOut = 0;
125         phaseOld = phase;
126         phase += deltaPhase;
127         if (phase == FRACT_MAX)
128         {
129                 phaseRemaining = FRACT_MAX - phaseOld;
130                 phase = deltaPhase - phaseRemaining;
131         }
132         switch (state){
133                 case 1:
134                         iChannel0RightOut = bitsr(sin2pi_01_fx16(phase)
                                )<<8;
135                         break;
136                 case 2:
137                         iChannel0RightOut = bitsr(square2pi_01_fx16(
                                phase))<<8;
138                         break;
139                 case 3:
140                         iChannel0RightOut = bitsr(saw2pi_01_fx16(phase)
                                )<<8;
141                         break;
142                 default:
143                         iChannel0RightOut = 0;
144         }
145
146 }
```

sin2pi_fx.c

```
1 #include <math.h>
```

```
#include "sin2pi_fx.h"

//-------------------------------------------------------------------------//
// Function:    sin2pi_pm_fx16                                             //
//                                                                         //
// Description: Maps the [-1,1) range into one period of the sine function.//
//-------------------------------------------------------------------------//
fract sin2pi_pm_fx16(fract x){
   if (x < -0.5r) {
           return -sin_fx16((x+FRACT_MAX)*2);
   } else if (x > 0.5r) {
           return sin_fx16((FRACT_MAX-x)*2);
   } else if (x == 0.5r) {
           return FRACT_MAX;
   } else {
           return sin_fx16(x*2);
   }
}

//-------------------------------------------------------------------------//
// Function:    sin2pi_01_fx16                                             //
//                                                                         //
// Description: Maps the [0,1) range into one period of the sine function.//
//-------------------------------------------------------------------------//
fract sin2pi_01_fx16(fract x){
    if (x < 0.25r) {                        /* < 0.25 */
        return sin_fx16(x*4);
    } else if (x < 0.75r) {                 /* < 0.75 */
        return -sin_fx16((-0.5r + x) *4);
    } else {
        return sin_fx16((FRACT_MIN + x) *4);
```

```
    }
}

fract square2pi_01_fx16(fract x){
        if (x < 0.5r) {
                return FRACT_MAX;
            } else {
                return FRACT_MIN;
            }
}

fract saw2pi_01_fx16(fract x){
        return x;
}
```

sin2pi_fx.h

```
#ifndef SIN2PI_FX_DEFINED
#define SIN2PI_FX_DEFINED

#include <stdfix.h>

fract sin2pi_pm_fx16(fract x);

fract sin2pi_01_fx16(fract x);

fract square2pi_01_fx16(fract x);

fract saw2pi_01_fx16(fract x);


#endif //SIN2PI_FX_DEFINED
```